Universidade POSITIVO

Desenvolvimento de Aplicativos Móveis

Aula 17 — Hooks: useEffect

Executando código quando a tela abre, quando algo muda — e limpando depois

Prof.: MSc. Alex Junior Nunes
pfsalexnunes@gmail.com

CONCEITO

O Que Falta no useState?

```
function TelaPerfil() {
  const [dados, setDados] = useState(null);

  fetch('https://api.exemplo.com/perfil')
    .then(r => r.json())
    .then(setDados);

  return <Text>{dados?.nome}</Text>;
}
```

O que acontece de verdade

Toda renderização chama fetch de novo — inclusive a renderização causada pelo próprio setDados. Isso dispara um looping de requisições, sem parar.

useState guarda um valor. Mas buscar dados, mexer em temporizadores, ouvir eventos — isso é código que precisa rodar em momentos certos, não a cada renderização.

CONCEITO

O Que É um Efeito Colateral?

Renderizar é "calcular o que mostrar na tela". Um efeito colateral (side effect) é qualquer código que sai desse cálculo e mexe em algo fora do componente.

- Buscar dados de uma API quando a tela abre
- Iniciar ou parar um temporizador (setInterval)
- Ouvir um evento (teclado, sensor, conexão)
- Salvar ou ler algo do armazenamento do aparelho

Já usamos essa ideia sem o nome: o useSelector do Redux e o useVideoPlayer da Aula 16 também lidam com coisas de fora do componente. useEffect é a ferramenta genérica pra isso.

Apresentando o Hook useEffect

```
import { useEffect } from 'react';

useEffect(() => {
  console.log('Componente montado!');
}, []);
```

1º argumento — a função de efeito

O código que você quer rodar fora do fluxo normal de renderização.

2º argumento — o array de dependências

Controla quando o efeito roda de novo. Vem na próxima slide.

useEffect é mais um Hook — mesma ideia do useState, do useSelector (Redux) e do useVideoPlayer (Aula 16): uma função especial, chamada no topo do componente.

O Array de Dependências

```
useEffect(fn)
```

sem array

roda depois de toda renderização — raro, cuidado com loops

```
useEffect(fn, [])
```

array vazio

roda uma única vez, quando o componente é montado

```
useEffect(fn, [valor])
```

array com valores

roda na montagem, e de novo sempre que valor mudar

Buscando Dados Quando a Tela Abre

```
useEffect(() => {
  fetch('https://api.exemplo.com/perfil')
    .then(r => r.json())
    .then((json) => {
      setDados(json);
    });
}, []);
```

O array vazio resolve o problema

Agora o fetch roda uma única vez, logo depois da primeira renderização — não a cada vez que o componente renderiza de novo.

Esse é o padrão mais comum de useEffect: buscar algo uma vez, assim que a tela é montada.

Combinando useEffect com useState

```
const [dados, setDados] = useState(null);
const [carregando, setCarregando] = useState(true);

useEffect(() => {
  fetch(url)
    .then(r => r.json())
    .then((json) => {
      setDados(json);
      setCarregando(false);
    });
}, []);

if (carregando) return <ActivityIndicator />;
return <Text>{dados.nome}</Text>;
```

Função de Limpeza (Cleanup)

```
useEffect(() => {
  const id = setInterval(() => {
    console.log('tic');
  }, 1000);

  return () => clearInterval(id);
}, []);
```

O que o efeito devolve

A função retornada (return) é a limpeza: roda antes do efeito repetir, e quando o componente é desmontado — desligando o que o efeito ligou.

Sem o clearInterval, o temporizador continuaria rodando mesmo depois da tela sumir — igual o expo-video, que precisa parar de tocar ao sair da tela.

Erro Comum: Esquecer o Array

ERRADO

```
useEffect(() => {
  setContador(contador + 1);
}); // sem array!
```

Sem array, o efeito roda depois de toda renderização. Ele muda um estado, o que dispara outra renderização — e o efeito roda de novo. Loop infinito.

CERTO

```
useEffect(() => {
  setContador(contador + 1);
}, []); // roda só uma vez
```

Com o array vazio, o efeito roda uma única vez. É quase sempre isso que você quer para inicializações.

Regra prática: se o efeito muda um estado usado dentro dele mesmo, confira o array com atenção — é a causa nº 1 de tela travada ou piscando sem parar.

Quando NÃO Usar useEffect

useEffect não é pra tudo. Um valor calculado a partir de outro estado não precisa de efeito — só deixa o código mais lento e complicado.

EVITAR

```
const [nomeCompleto, setNomeCompleto]
  = useState('');

useEffect(() => {
  setNomeCompleto(`${nome} ${sobrenome}`);
}, [nome, sobrenome]);
```

PREFERIR

```
const nomeCompleto =
  `${nome} ${sobrenome}`;
```

Calculado direto no corpo do componente, a cada renderização — sem estado extra, sem efeito, sem atraso de um render.

DESAFIO

Desafio da Aula: Lista de Alunos

Em duplas, montem uma tela que busca uma lista ao abrir:

- [ ] useState pra guardar a lista de alunos e o estado de carregando
- [ ] useEffect com array vazio, buscando os dados uma única vez
- [ ] Enquanto carregando é true, mostra "Carregando..." na tela
- [ ] Depois, mostra a lista de alunos com FlatList

`Pasta: 07-desafio-lista-alunos/  ·  Gabarito: 07-desafio-lista-alunos-solucao/`

O Que Levamos de Hoje

- useEffect roda código fora do fluxo normal de renderização: efeitos colaterais
- O array de dependências decide quando o efeito roda: sem array, [], ou [valores]
- Array vazio ([]) é o padrão pra buscar dados uma vez, ao montar a tela
- A função de limpeza (return) desliga o que o efeito ligou — temporizadores, listeners
- Nem todo cálculo precisa de useEffect — só o que realmente sai do fluxo de renderização

Próxima aula: seguimos avançando no cronograma — conteúdo definido de acordo com o ritmo da turma.